

Kunming AI+

An Applied Introduction to AI with PyTorch

Luca Ghafourpour



**UNIVERSITY OF
CAMBRIDGE**
Department of Engineering

An Introduction



BSc in Aerospace Engineering – Advisor Steven Hulshoff



Motorsport aerodynamicist and Formula E Data engineer



MSc in Computational Methods for Aeronautics – Advisor Sylvain Laizet



MSc in Statistics – Advisor Alessio Figalli



ML Researcher – Advisor Anima Anandkumar



PhD in Advanced Machine Learning – Advisor Rich Turner

The Programme

AIM: To be able to build, train, and diagnose your own neural networks in PyTorch, and gain an appreciation for the current frontier of AI in the sciences

Session Title	Topics
Workshop 1	VS Code, Conda and PyTorch
Practical Workshop 1	Constructing and training a neural network from scratch
Workshop 2	PyTorch modules for building and training neural networks
Practical Workshop 2	PyTorch modules for data loading
Workshop 3	Diagnosing training performance
Practical Workshop 3	Initialisation and regularisation
Academic Lecture 1	Physics Informed Neural Networks
Academic Lecture 2	Neural Operators
Workshop 4	Convolutional neural networks
Practical Workshop 4	Scikit Learn
Workshop 5	Complete end-to-end experiment
Practical Workshop 5	Open Q&A

Workshop 1



<https://www.menti.com/aleqakdb9v4h>

What is an IDE?

Integrated Development Environment is a single application that bundles every tool a developer needs.

Think of it as a workbench

Instead of switching between a text editor, a terminal, a debugger and a browser, the IDE puts them all into one application. You write code, run it, find bugs, and deploy without leaving the window.

Popular picks for Python

VS Code · PyCharm · Spyder · JupyterLab



Smart editor

Syntax highlighting, autocomplete, refactoring.



Debugger

Pause execution, inspect variables, step through code.



Runner & terminal

Execute scripts and open a shell inside the IDE.



Version control

Built-in Git for commits, branches, and diffs.

How an IDE works with Python

The IDE talks to a Python interpreter behind the scenes.

The pipeline

1

You write .py code

in the editor

2

IDE sends it to the interpreter

configured per project

3

Interpreter executes

output streams back to the IDE

4

You inspect & debug

variables, stack, breakpoints

hello.py

```
def greet(name):  
    return f"Hello, {name}!"  
  
print(greet("world"))
```

Terminal

```
$ python hello.py
```

```
Hello, world!
```

```
Process finished with exit code 0
```

What is Conda?

A package & environment manager.

Two jobs, one tool

- 1. Installs packages:** NumPy, pandas, PyTorch, even non-Python libraries like CUDA.
- 2. Manages environments:** isolated workspaces, each with its own Python version and packages.

Which one do I install?

Miniconda

RECOMMENDED

Lightweight (~400 MB). Just conda + Python. You install only what your project needs.

Anaconda Distribution

Heavyweight (~3 GB) bundle with ~250 data-science packages preinstalled. Convenient but bulky.

Installing Miniconda via CLI

Three commands on Linux/macOS. Windows users: grab the .exe installer from the same page.

01 Download

Fetch the installer for your OS. The latest link is on docs.anaconda.com.

```
$ curl -O https://repo.anaconda.com/miniconda/Miniconda3-latest-Linux-x86_64.sh
```

02 Run installer

Execute the shell script. Accept the licence, then choose an install path.

```
$ bash Miniconda3-latest-Linux-x86_64.sh  
  
# follow the prompts
```

03 Verify

Reload the shell so conda is on your PATH, then check it works.

```
$ source ~/.bashrc  
$ conda --version
```

Tip: during the install, answer **yes** when asked to run **conda init**. It wires conda into your shell automatically.

Why separate environments?

One global Python install will lead to broken projects. Isolation keeps them from conflicting with each other.



Without separate envs

✗ Version clashes

Project A needs pandas 1.5, B needs pandas 2.2.

✗ “Works on my machine”

Your interpreter has packages teammates don't, and vice versa.

✗ Hard to reproduce

No clean record of what's actually installed.

✗ System pollution

Globally installed packages slowly turn into a junk drawer.



With separate envs

✓ One project, one env

Each project gets its own Python + packages, fully isolated.

✓ Pin versions safely

Lock pandas 1.5 in one env, 2.2 in another. Both coexist.

✓ Reproducible

Export to environment.yml, meaning others can exactly rebuild.

✓ Disposable

Broken env? You can delete and recreate. Your system stays clean.

Key Conda commands

The everyday vocabulary. Memorise these and you're 90% there.

conda create -n myenv python=3.11

Create a new environment named myenv with Python 3.11.

conda env list

Show every environment on the machine.

conda activate myenv

Switch into the environment. Your prompt updates.

conda list

List packages installed in the active env.

conda deactivate

Leave the current env, return to base.

conda env export > environment.yml

Snapshot the env to a shareable YAML file.

conda install numpy pandas

Install packages into the active environment.

conda env remove -n myenv

Delete an environment completely.

Recap

Three things to remember

1 **An IDE is your workbench.**

Editor, runner, debugger, version control, all in one window.

2 **Conda manages packages and environments.**

Install Miniconda unless you specifically want the full Anaconda bundle.

3 **One environment per project.**

Create, activate, install, export. Repeat for every new project.